EXPERIMENT 8

AIM

Configuring a Cryptocurrency Wallet using MetaMask and integrating it with a DApp.

THEORY

1. Cryptocurrency Wallet

A cryptocurrency wallet is a software application or digital tool used to manage blockchain accounts and interact with blockchain networks. A wallet does not physically store cryptocurrency. Instead, it manages the private keys that allow a user to access and authorize transactions associated with their blockchain address. A typical Ethereum wallet provides: Ethereum wallet address, private key management, account balance, transaction signing, connection with decentralized applications, and interaction with smart contracts.

2. MetaMask

MetaMask is a cryptocurrency wallet and browser extension that allows users to interact with Ethereum-compatible blockchain networks and decentralized applications. It acts as a bridge between: User → MetaMask → Blockchain / Smart Contract → DApp. MetaMask creates and manages Ethereum accounts, stores and manages cryptographic keys, displays wallet balances, allows users to approve blockchain transactions, allows DApps to request access to the user's wallet, signs transactions before they are submitted to the blockchain, and supports Ethereum test networks such as Sepolia.

3. Ethereum Network

Ethereum is a blockchain platform that supports programmable transactions through smart contracts. For development and testing, a test network such as Sepolia can be used instead of the Ethereum mainnet. The advantage of using a test network is that developers can test wallet connections, smart contracts, transactions and DApp functionality without using real cryptocurrency.

4. Ethereum Wallet Address

Every Ethereum account has a unique wallet address used to identify the account on the blockchain. The address can be used to receive ETH, send ETH, interact with smart contracts, and identify the account performing a blockchain transaction. When a DApp connects to MetaMask, it can obtain the user's wallet address after the user grants permission.

5. DApp

A Decentralized Application (DApp) is an application that interacts with blockchain-based infrastructure rather than relying completely on a centralized backend. A typical DApp consists of: Frontend → Web3 provider / Wallet → Smart Contract → Blockchain. In the Ethlance implementation: the frontend provides the user interface, MetaMask provides wallet connectivity, Ethereum Sepolia provides the blockchain network, smart contracts provide blockchain-based functionality, and transactions are authorized using the user's wallet.

6. Wallet–DApp Integration

The basic process of integrating MetaMask with a DApp is: (1) open the DApp in a browser; (2) install/open MetaMask; (3) select the required Ethereum network; (4) select the required wallet account; (5) click Connect Wallet on the DApp; (6) MetaMask displays a connection request; (7) the user approves the request; (8) the DApp obtains access to the wallet address; (9) when a blockchain operation is performed, the DApp requests a transaction through MetaMask; (10) the user reviews and confirms the transaction; (11) MetaMask signs and submits the transaction to the blockchain; (12) the transaction is recorded on the blockchain.

7. Gas Fees

Blockchain transactions require computational resources from the Ethereum network. The associated transaction cost is called gas. Gas is required for operations such as deploying smart contracts, calling functions that modify blockchain state, transferring ETH, and performing other blockchain transactions. In a test network such as Sepolia, test ETH can be obtained from a faucet and used to perform these operations without spending real ETH.

8. Smart Contracts

A smart contract is a program deployed on a blockchain that automatically executes predefined logic when its conditions are satisfied. In the Ethlance DApp, smart contracts are used to implement functionality related to the decentralized freelance marketplace: EthlanceStructs and JobHelpers are libraries, Job is the job contract, MutableForwarder and EthlanceProxy are proxy contracts, and Ethlance is the main marketplace implementation, configured through the proxy and initialized with the deployed Job contract.

PROCEDURE

Opened the MetaMask browser extension (already installed in the Chrome browser used for this experiment).

Configured the Ethereum wallet account in MetaMask (account “Afnan Kazi”, 0xd682b97bd00f4de6ba5d19b4181377b3acf5f74c, set up during the earlier blockchain experiments and reused here).

Added/configured the Ethereum Sepolia Test Network.

Obtained test ETH from the Sepolia faucet (0.05 SepoliaETH in the earlier experiment; the wallet held 0.01893877 SepoliaETH when this experiment began).

Verified the test ETH balance in MetaMask (Fig. 1).

Opened the Ethlance DApp at ethlance.com.

Selected the option to connect the cryptocurrency wallet.

MetaMask displayed a permission request to connect the account (Fig. 2).

Approved the connection request; the Ethlance DApp was connected to the MetaMask wallet (Fig. 3).

Opened the local Ethlance deployment page (http://localhost:8081/) and connected the same MetaMask account on Ethereum Sepolia (Fig. 4).

The Ethlance implementation was deployed on the Sepolia network from the deployment page; MetaMask was used to approve each deployment transaction (Fig. 5).

The deployment completed successfully: all six contracts were deployed, the proxy was configured with the Ethlance implementation, and the implementation was initialized with the Job contract (Fig. 6).

The deployed DApp contracts were verified on Sepolia Etherscan (Fig. 7).

Verified that the DApp can interact with the blockchain through the configured MetaMask account: the proxy's target() reads the Ethlance implementation and jobProxyTarget() reads the Job contract.

CODE AND OUTPUT

Smart Contracts Deployed (Ethlance, district0x/ethlance)

The Ethlance contracts were compiled from source with Solidity 0.8.17 (optimizer enabled, runs = 1) using OpenZeppelin Contracts 4.1.0, exactly as specified by the project's truffle configuration. None of the six contracts takes constructor arguments. The two configuration calls of the system are:

```
// EthlanceProxy.sol - point the proxy at the Ethlance implementation
function setTarget(address _target) public auth {
    target = _target;
}

// Ethlance.sol - initialize the implementation through the proxy
// (stores the address of the contract that Job proxies delegate to)
function initialize(address _jobProxyTarget) external {
    require(_jobProxyTarget != address(0));
    this.setJobProxyTarget(_jobProxyTarget);
}
```

Deployment Script (local deployment page, ethers.js v6 + MetaMask)

The local deployment page deploys the contracts in dependency order, links the libraries into Job and Ethlance, then configures the proxy:

```
const DEPLOY_ORDER = ["EthlanceStructs", "JobHelpers", "Job",
                      "MutableForwarder", "EthlanceProxy", "Ethlance"];
// 1. Deploy EthlanceStructs and JobHelpers (libraries)
// 2. Deploy Job, linked with EthlanceStructs + JobHelpers
// 3. Deploy MutableForwarder and EthlanceProxy
// 4. Deploy Ethlance, linked with EthlanceStructs
// 5. EthlanceProxy.setTarget(Ethlance)
// 6. Ethlance.at(EthlanceProxy).initialize(Job)
```

Deployment Details (verified on Sepolia)

Wallet (MetaMask account “Afnan Kazi”): 0xd682b97bd00f4de6ba5d19b4181377b3acf5f74c. EthlanceStructs: 0xC2C51e89D94DB33695675d0D0722c0DFCF7f5f85 (tx 0x7677a02075186cb69b5095cb7807397264a0dd3f4ec2a36319cdd0af34779f76, gas used 10,820,980). JobHelpers: 0x52a5A83840850856Cbf22a344C568DC96e072F02 (tx 0x3b4faf83d8abe4270fe37d0851d48c3e51501cbdfdb7e609dab310300ee528fd, gas used 2,712,779). Job: 0x5eA40d87EBDcD9473edbe8C6Df9466a6D9562027 (tx 0x6fd80966abda7f57c6419aea5a022b14877530b0bdf921180f04536b86a736d3, gas used 32,951,534). MutableForwarder: 0x9AdCc733Ccc8f040A3168ec75cac3754b3B6A045 (tx 0xf01c5680a51630d96fb41ec603b4c460fe5c788fa28d52461f2b13b5b975391f, gas used 2,281,634). EthlanceProxy: 0x2d3b68e3f8F2359f74EdD005C83243c89767c45F (tx 0x40680452c3252feaa8eccc45f5066091e4d0ad1e6af4832448712fdf82dd62cd, gas used 2,264,664). Ethlance: 0xA02F0A03ed023877A500D3e7D886805F11Ef3b05 (tx 0x3ba0d76ae493f5833ca868b749044afc643a44f51114f8cff8698cbb864097eb, gas used 15,888,067). EthlanceProxy.setTarget(Ethlance): 0xb9938aced5d847f24fc282ab29075e7c4e9ab81c17ef767b125534c7a53509ed (gas used 30,274). Ethlance.initialize(Job) through the proxy: 0xfefe1a4a48d4d2c3abd40edaffd297fd2ae0f1cc45c8e1bb937979e99a29dfb7 (gas used 134,061). On-chain verification: EthlanceProxy.target() returns the Ethlance implementation and jobProxyTarget() returns the Job contract. Wallet balance before deployment 0.01893877 SepoliaETH, after deployment 0.01123037 SepoliaETH (total spent 0.00770840 SepoliaETH, including one failed first attempt: an EthlanceStructs creation, tx 0xa364ac26e4183a7c5b7ec70907bd82d84e1608866fe64a4c2809ff5b2a1d3bce, which ran out of gas at a 2,000,000 gas limit and cost 0.001 SepoliaETH; the limits were then raised and the deployment succeeded).

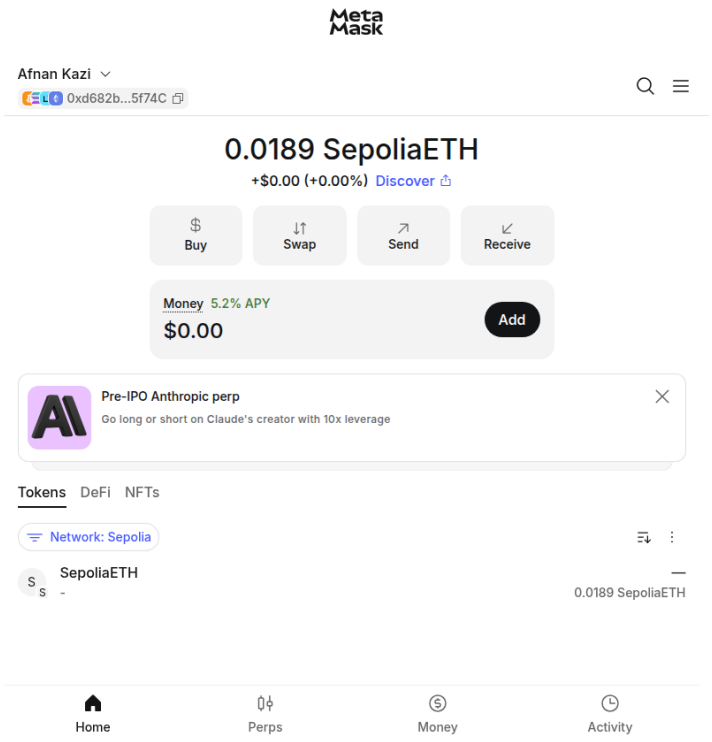


Fig. 1. MetaMask wallet - account “Afnan Kazi” on the Sepolia network with 0.0189 SepoliaETH

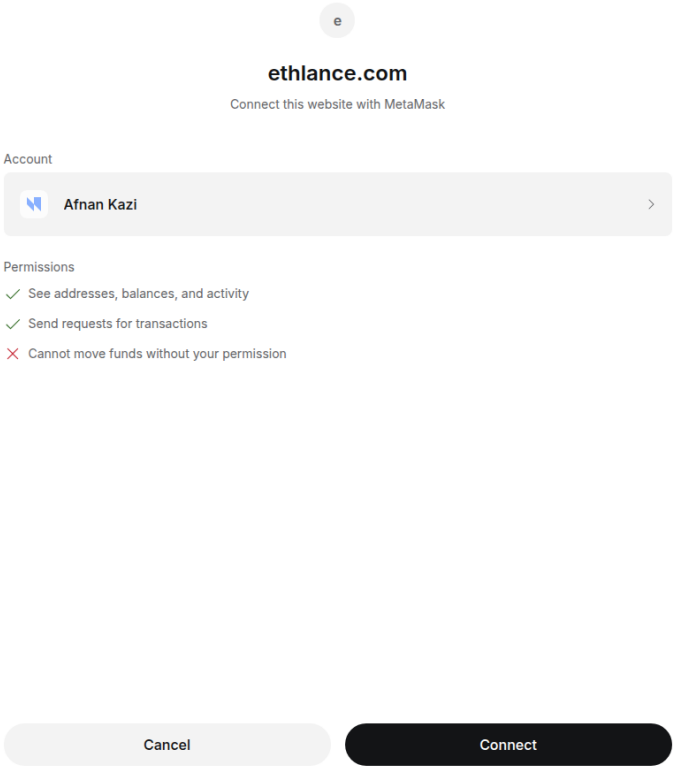


Fig. 2. MetaMask connection request from ethlance.com

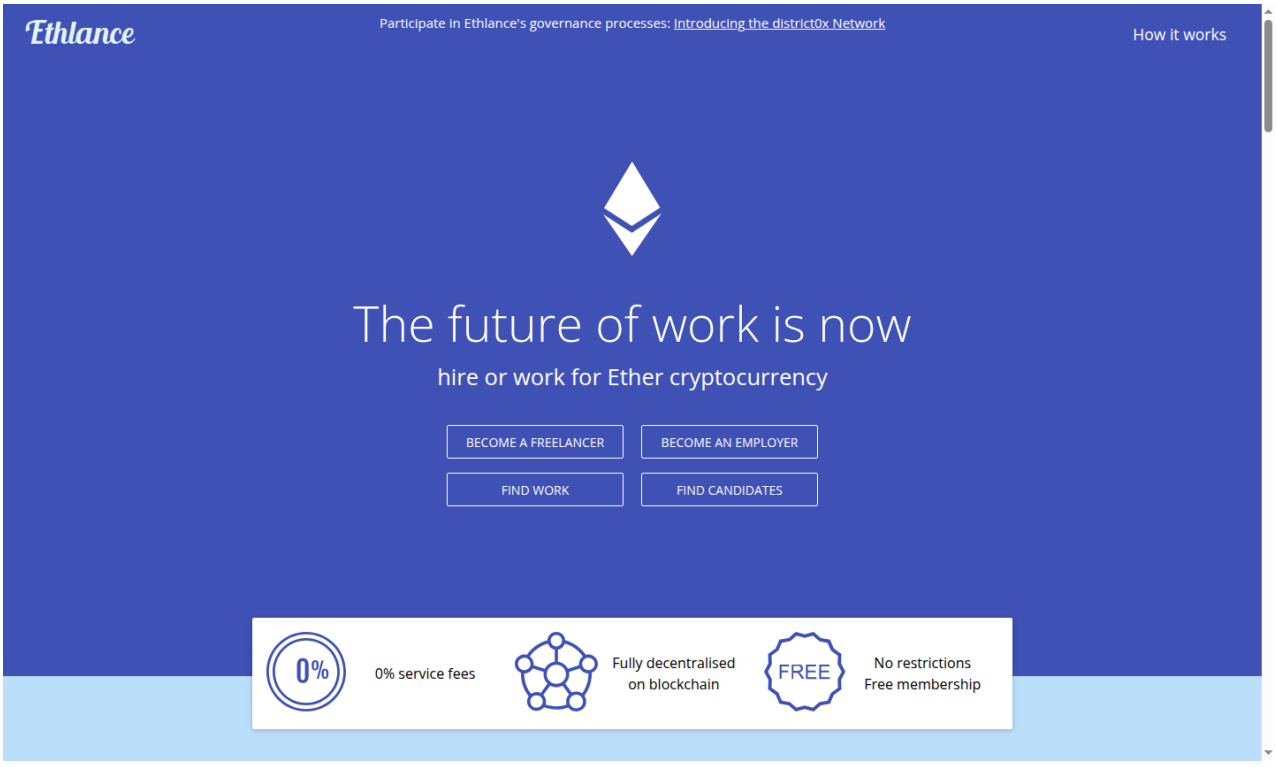


Fig. 3. Ethlance DApp (ethlance.com) after the wallet was connected

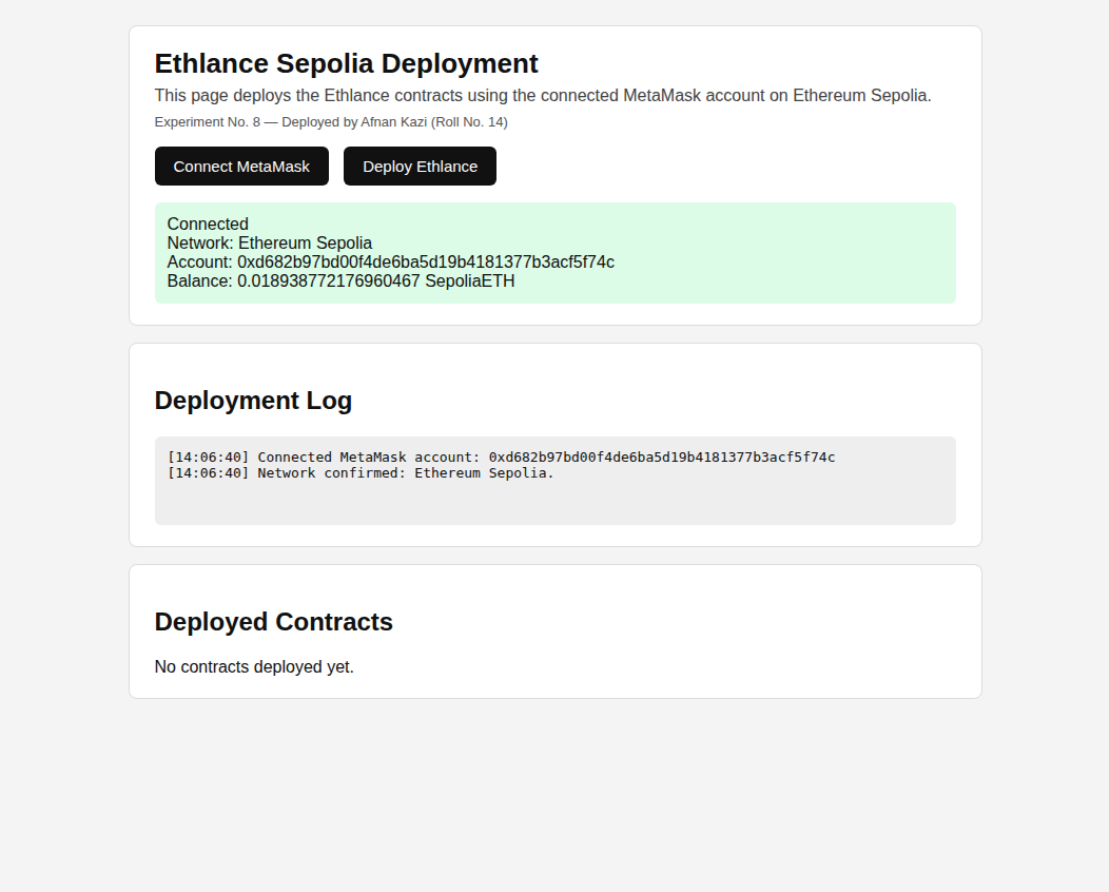


Fig. 4. Local Ethlance deployment page connected to MetaMask on Ethereum Sepolia

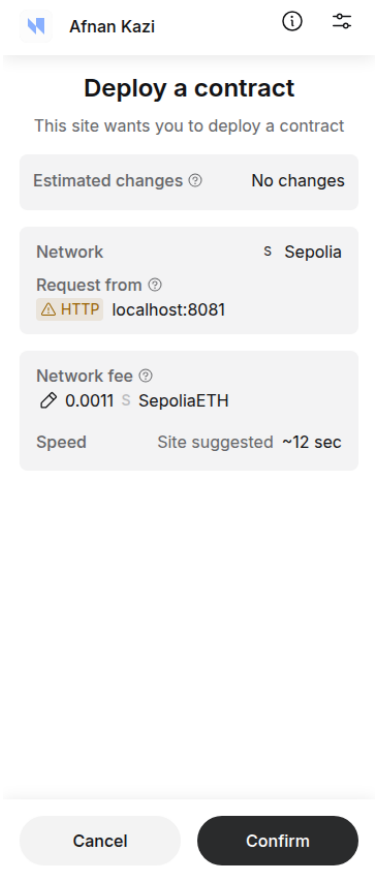


Fig. 5. MetaMask confirmation for deploying a contract from the local deployment page

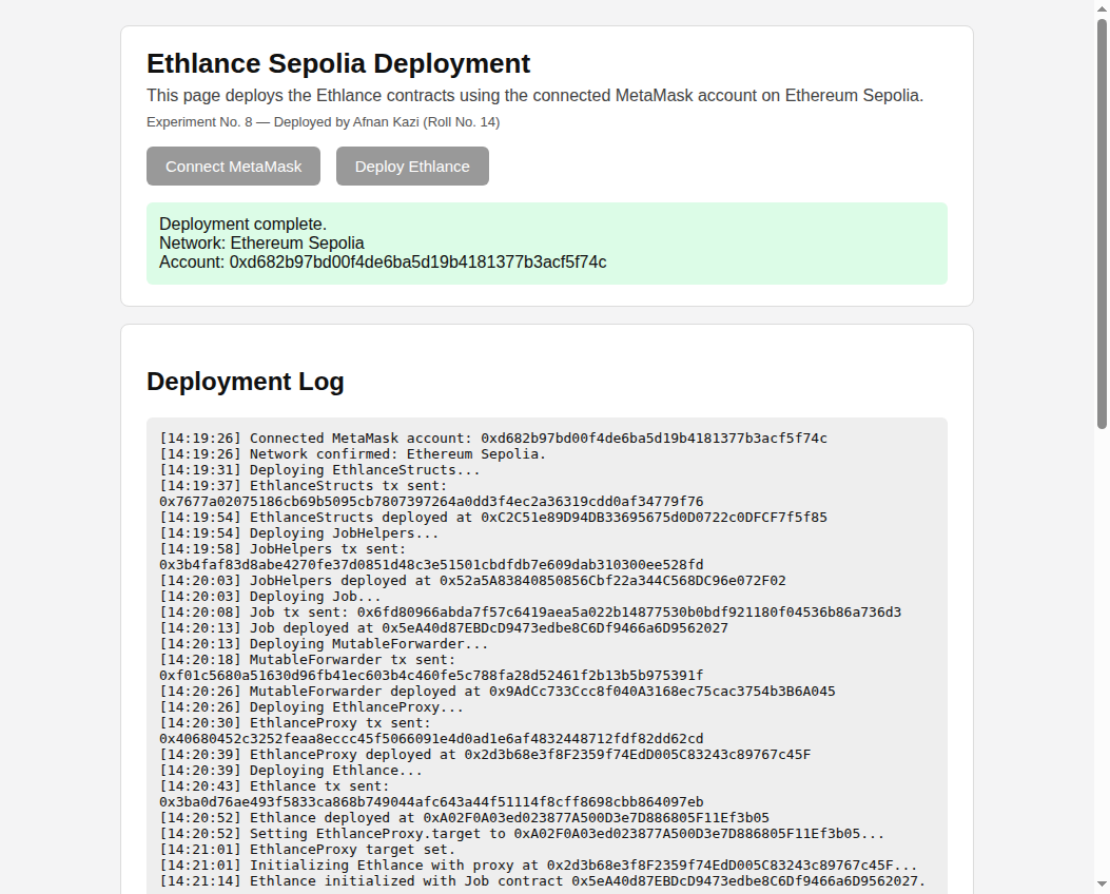


Fig. 6. Deployment complete - log of all six contracts, proxy configuration and initialization

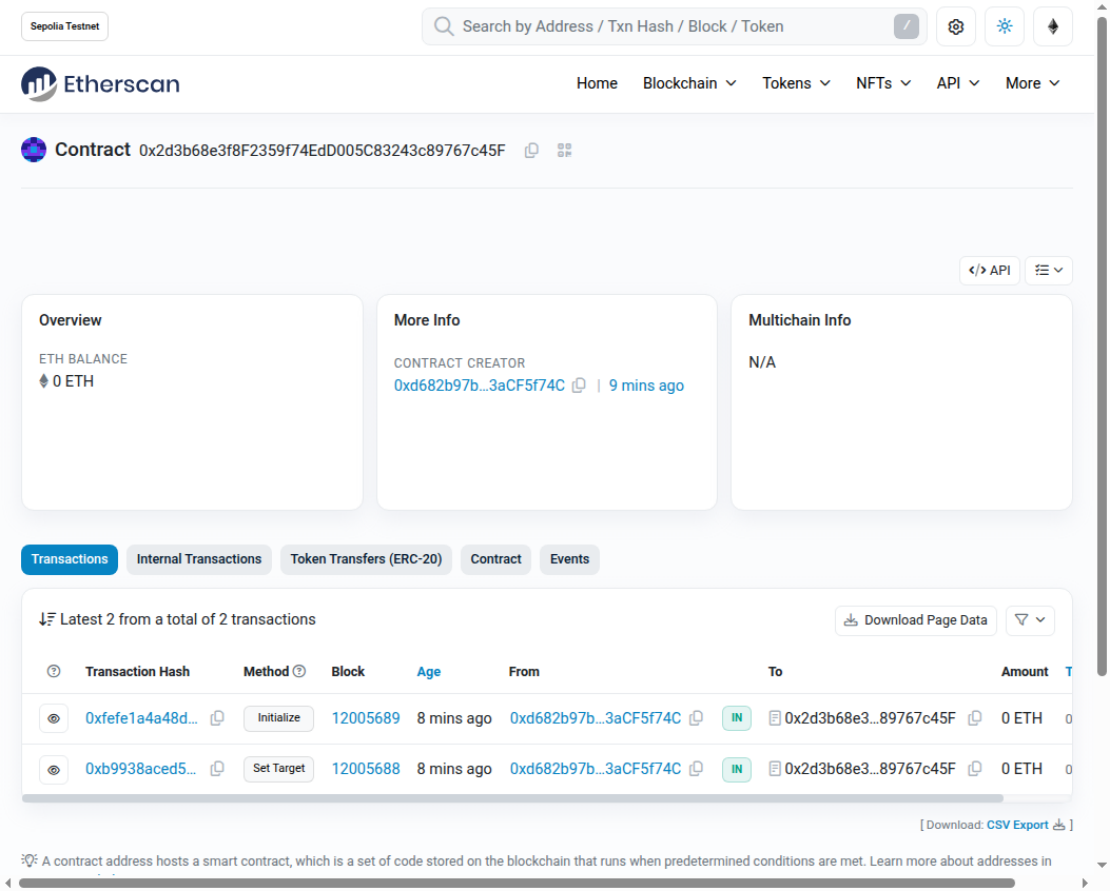


Fig. 7. Sepolia Etherscan verification of the EthlanceProxy contract

Note on Fig. 3: the live ethlance.com homepage is a legacy interface and does not display the connected wallet address on the page itself; the connection is shown by the approved MetaMask prompt in Fig. 2 and was verified in the browser (eth_accounts returned the wallet address and the site's “No accounts connected” indicator disappeared).

CONCLUSION

Thus, the MetaMask cryptocurrency wallet was successfully configured and connected to the Ethereum Sepolia test network. Test Ether was obtained and used for blockchain transactions. The Ethlance decentralized application was successfully integrated with MetaMask, allowing the wallet to authorize interactions with the Ethereum blockchain and deployed smart contracts: the six Ethlance contracts were deployed on Sepolia, the EthlanceProxy was configured with the Ethlance implementation, and the implementation was initialized with the Job contract, all verified on Sepolia Etherscan. The experiment demonstrated how cryptocurrency wallets act as an interface between users and DApps, enabling secure transaction signing and blockchain-based application interaction.